

dry

1. Steps

1. **We'll install a jump at the start of poll.**

This jump redirects execution to our hook. Since poll writes to a global, the memory is writable, making this safe.

2. **We'll place our hook code in a new section at the end of the file.**

This gives us space to write clean logic without interfering with existing code.

3. **We'll recreate the overwritten prologue of poll inside the hook.**

This ensures the stack and control flow remain intact as if nothing changed.

4. **We'll write our custom interval value into the global variable.**

This replaces the original behavior, letting us set any delay we want.

5. **We'll jump back into poll after its original memory write.**

This skips the part where poll would have overwritten our value.

Key Points

- The memory is already writable.
- Our hook runs every time poll executes.
- We skip the original write to avoid being overwritten.
- No flags or assumptions are needed.
- We fully control where execution resumes.
- We assume the global write occurs before any return in poll.
- The return jump target must be chosen carefully by analyzing instruction boundaries.

The hook changes the polling interval reliably by injecting early and skipping over the original assignment.

2. Steps

1. **Discover every call-target inside mine** using symbol analysis or linear sweep. Each target must be treated as a separate entry point to hook, as shown in the "Hot Patching" technique.
2. **Overwrite the first 5–15 bytes at each entry with a relative jmp to a shared trampoline.** The trampoline will include the original overwritten instructions to preserve execution flow.
3. **In the trampoline:**
 1. Save volatile registers and flags.
 2. Locate the input parameter ([esp+4], ecx, or rcx/rdi depending on platform) and apply not.
 3. Restore registers.
 4. Jump back to the original function after the overwritten bytes.
4. **Use the end of mine (optional tail patch)** if additional space is needed. The problem permits overwriting the single ret, allowing extra hook logic to be stored there.
5. **Ensure ASLR- and DEP-safety** by resolving runtime addresses via GetModuleHandle and GetProcAddress. If no safe code caves are available, use VirtualAlloc to allocate executable memory, as done in DLL-injection setups.

Key Points

- Handles multiple entry points explicitly, avoiding reliance on a single prologue hook.
- Leaves the single return point untouched, preserving expected exit behavior.
- Does not attempt to restore x, as the goal is to compute $f(\sim x)$ — reinversion is unnecessary and would corrupt the result.
- Assumes at least 5 writable and executable bytes at each hook location; fallback stubs may be needed if space is tight.
- Argument handling depends on the calling convention; support for fastcall and full 64-bit ABIs requires explicit extension.
- Not fully memory-free — although in-place hot patching is possible, external space allocation may be needed in constrained binaries.

3. Steps

1. **Locate the IAT entry for sendf.**

Use PE parsing to find the Import Address Table and identify the pointer for sendf.

2. **Replace the IAT entry with your hook function.**

Use VirtualProtect to make the IAT writable, then patch the function pointer to redirect to your hook.

3. **Implement the hook function.**

Accept the original arguments (socket, format string, and variadic args). Use va_list, va_start, and vsnprintf to format the message.

4. **Encrypt the formatted buffer.**

Apply encryption to the resulting message.

5. **Call the original sendf using the same format string and arguments.**

To preserve format fidelity, do not replace the format string with "%s"—instead, pass the original format and args to vsnprintf, then encrypt the output and send it as a raw message.

6. **Optionally wrap the original format call.**

If reformatting is necessary, preserve argument count and types by parsing the format string and forwarding them accordingly.

7. **Manage memory and restoration.**

Free buffers after use. If restoring the IAT is required, ensure thread safety using atomic or lock-free operations.

Key Points

- IAT hooks only affect external calls; internal/static calls to sendf are not intercepted.
- Replacing the format string with "%s" is lossy—maintain the original format and argument structure to avoid breaking downstream behavior.
- Ensure proper handling of variadic arguments to avoid stack or heap corruption, especially under cdecl.
- Consider fallback to hot patching if full control over all call sites is required.
- Thread safety must be explicitly handled when patching or restoring IAT entries.

4. Steps

1. Resolve Function Addresses

At runtime, use `GetModuleHandle()` and `GetProcAddress()` to determine the entry addresses of both `parse` and `connect`.

2. Patch Each Function Prologue

For each of the two functions:

- Read and save the first N bytes of the prologue (where $N \geq 5$).
- Use `VirtualProtect()` to enable writing to the function's code page.
- Overwrite the prologue with a relative `jmp` to a trampoline.
- Restore the original memory protection.

3. Implement a Shared Trampoline

In an RWX memory region (e.g., via `VirtualAlloc()`):

- Save registers and flags (`pushad` / `pushfd`).
- Execute the saved prologue bytes to preserve original logic.
- Capture arguments into a struct and call `CreateThread()` with a worker that:
 - Receives the arguments,
 - Calls the rest of the original function (`entry + N`),
 - Exits via `ExitThread()`.
- Restore the original state (`popfd` / `popad`).
- Jump back to `entry + N`, skipping the rest of the function body in the main thread.

4. Thread Management

After both hooks execute:

- Use `WaitForMultipleObjects()` if the original logic depends on the results of `parse` or `connect`.
- Always `CloseHandle()` on thread handles to avoid resource leaks.

Key Points

- Each function receives its own patch via a single `jmp` at its prologue, avoiding complex control-flow hacks.
- Volatile registers and flags must be saved and restored to preserve correct behavior.
- We can ensure each function's prologue is at least 5 bytes long to accommodate a `jmp` instruction; otherwise, we can use a larger trampoline or adjust patching strategy.
- All addresses are resolved dynamically and patched memory is safely updated using `VirtualProtect`, making the solution ASLR- and DEP-safe.
- Argument data must be copied into a thread-safe structure before thread creation to avoid races with stack/register reuse.

- When using a shared trampoline, we can include logic to distinguish which function triggered it (e.g., by comparing the return address or thread-local identifiers).
- Store each thread handle in a global or thread-local array immediately after `CreateThread()` for proper cleanup and synchronization.
- We can use `WaitForMultipleObjects()` whenever subsequent code depends on the results of `parse` or `connect`—synchronization is mandatory in such cases to avoid data races.

5. Steps

1. **Locate and validate calc**

Use `GetModuleHandle` and `GetProcAddress` to find `calc`. Disassemble its prologue to ensure at least 5 bytes can be overwritten. If not, prepare a thunk and use a long jump.

2. **Set up infrastructure**

Allocate an RWX trampoline, store original prologue bytes, and pre-build the jump back. Initialize a `CRITICAL_SECTION` for safe patch toggling. Set up a lock-free log buffer and a background flusher thread.

3. **Apply hot patch**

Temporarily unprotect the code page, write a 5-byte `jmp` at the start of `calc`, and restore protection.

4. **Trampoline logic**

Save volatile state, enter critical section, restore original prologue, call `calc + N`, reapply the hook, log return value, restore state, and return to caller.

5. **Handle integrity checks**

If a watchdog might scan `calc` mid-call, delay its checks via a timer thread. If this fails, fallback to IAT hooking.

6. **Optional fallback**

If `calc` is imported, patch its IAT entry instead of the prologue.

7. **Cleanup**

On process exit, flush logs, close handles, and delete the critical section.

Key Points

- Only one `jmp` is written inside the target binary (at `calc`'s entry).
- Thread safety is ensured via a critical section; recursion is not supported.
- The prologue is temporarily unhooked during execution, which may expose the patch.
- The IAT fallback applies only if `calc` is imported.
- If prologue is too short, use a thunk and long jump instead.

6. Steps

1. Verify patch feasibility

- Disassemble the first instructions of solve; ensure at least 5 contiguous bytes can be overwritten.
- Allocate an executable code cave for two trampolines (entry and tail).

2. Entry hot-patch

- Overwrite the prologue with one relative jmp to the **entry-trampoline**.
- Store the original bytes for later restoration.

3. Entry-trampoline actions

- Save **all** volatile and non-volatile registers plus flags (x86 / x86-64).
- Increment a **thread-local** recursion counter.
- Restore the original prologue, then jump to solve+N so the real body executes intact.

4. Discover and patch every exit (first execution only)

- While control flows through the body, a lightweight disassembler records every ret/retn imm8.
- Replace each recorded return with a short jump to a shared **tail-trampoline**.

5. Tail-trampoline logic

- On arrival from any return site, capture the value in EAX/RAX.
- Decrement the thread-local counter.
- Double the value **always** (case a) or **only when the counter is zero** (case b).
- Place the adjusted value back into the return register.
- Execute an exact copy of the original ret instruction to satisfy CFI/CET and return to the caller.

6. Thread-safe patch management

- Guard patch/un-patch operations with a CRITICAL_SECTION wrapped in SEH (__try / __except) so locks are released on unexpected faults.

7. Integrity & security considerations

- Because the body runs with its original code, most self-checksums pass; if tail bytes are covered by a checksum, delay that check with a timer until after the tail patch is restored.
- Resolve all addresses at runtime and toggle page protections with VirtualProtect to remain ASLR/DEP compliant.

8. Cleanup

- On process detach, restore every patched return and the original prologue, free the trampolines, and delete the critical section.

Key Points

- Single if (depth==0) test in the tail-trampoline switches between “double every call” (**case a**) and “double only the outermost call” (**case b**).
- **All exits captured** – dynamic scan patches every return instruction, avoiding the “single-ret” assumption.

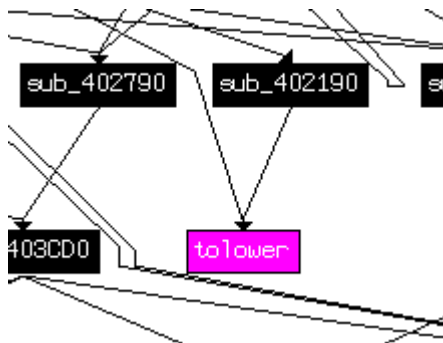
- **One-jmp rule respected** – only one detour at the prologue; exit patches remain inside the same function and do not add extra hooks elsewhere.
- **Thread-local depth counter** – prevents races in multi-threaded recursion; counter balance is guaranteed because the tail-trampoline runs on every exit path.
- **CFI/CET safe** – tail-trampoline ends with the exact original ret opcode, bypassing control-flow-integrity checks.
- **SEH-protected critical section** – avoids deadlocks if exceptions occur while the code is writable.
- **Full register preservation** – includes non-volatile registers (RBX, RBP, etc.) for 64-bit builds.
- **Minor residual risks acknowledged**
 - **Disassembly overhead** – initial exit-map construction adds one-time latency but is negligible for long-running processes.
 - **Deep recursion** – extremely deep call trees could overflow the thread-local counter; choose a wide integer type or add overflow checks if this is a concern.
 - **Tail-byte checksums** – if a security routine hashes the bytes around each ret, an additional stub may be required to spoof the expected hash.
 - **Unusual calling conventions** – if solve uses a non-standard convention (e.g., returning via EDX:EAX), adjust the tail-trampoline to handle multi-register returns.

Part 1

First im trying to run keygen.exe with input string:

```
• PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "A"  
|  
• PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "ABCD"  
|3Jj|  
• PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "ABCD A"  
|3Jj|
```

Tried opening in IDA to look for scanf function of some sort. Found 'tolower' function:



So maybe output is the same for lower/uppcase.

```
• PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "ABCD A"  
|3Jj|  
• PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "abcda"  
? $bh?
```

It seems like each character is mapped to a single output (ex. 'A' always maps to '|').

We probably need to create a mapping for each character.(for loop in python file which saves output of each character after running keygen with the unique input.

```
keygen_mapping.txt  
1 'a': ?  
2 'b': $  
3 'c': b  
4 'd': h  
5 'e':  
6 'f': /  
7 'g': M
```

```
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen "AIE"
| (
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen "AeE"
| (
```

We continue to make a reverse mapping for the .txt file to be able to get the input which gave us the output.

```
PS C:\Users\TA\Desktop\hw3\20752> Enter the mapped string: ABCD
Original string: $/VD
```

They mention hw1's passwords so we figured we need to use reverse keygen on one of those.

הסיסמא שלך היא (ללא רווחים):

Didn't work.

```
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "8"
```

```
keygen_mapping.txt
7 '0': D
8 '1': R
9 '2': F
0 '3': k
1 '4': z
2 '5': w
3 '6': ]
4 '7': Z
5 '8': Y
6 '9': g
7 ' ': .
8
```

Searching for all possible inputs manually to try and find which one gives 8 as output we found out that '*' gives 8x where x is a printable character.

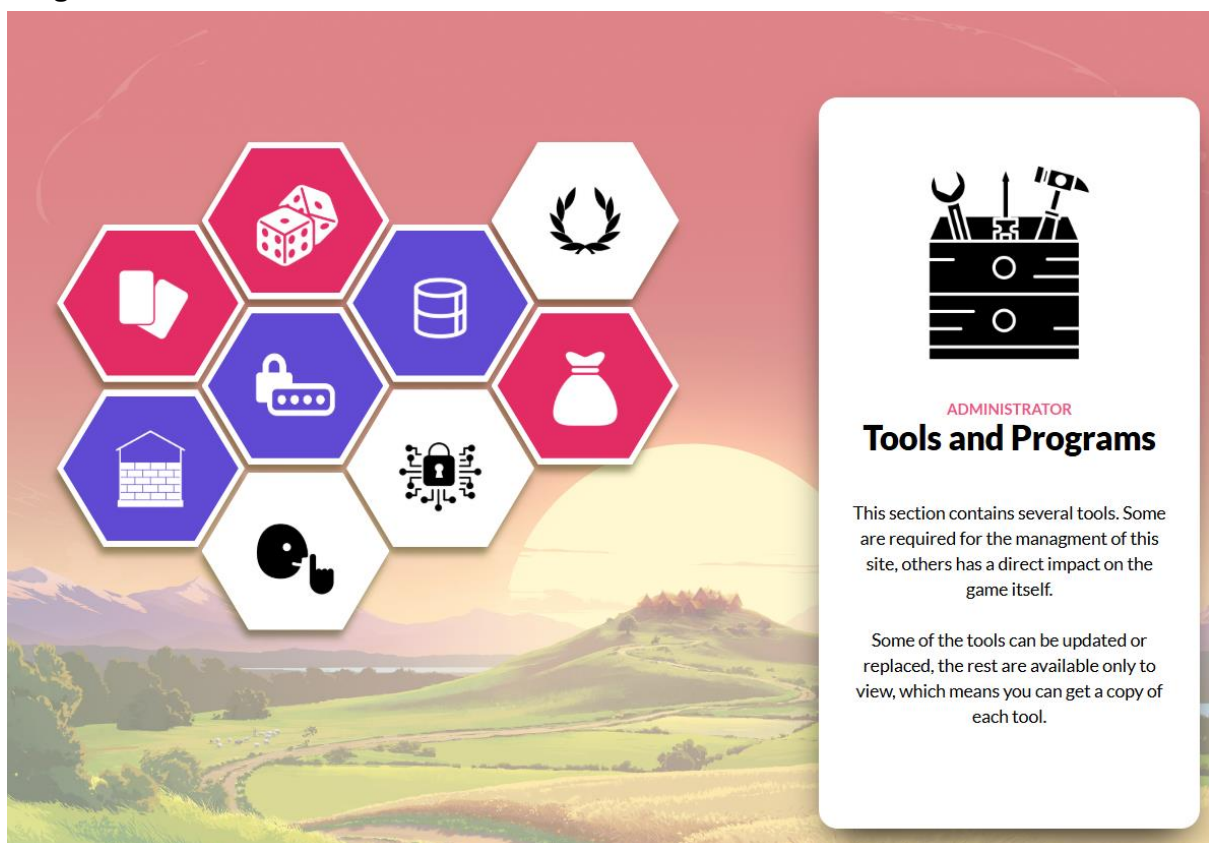
Searching for values we found that *I (which we previously found out is space of some sort) gives us 8 as output:

```
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./keygen.exe "*"I"
8
```

So we run the reverse mapper and get:

```
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> & C:/Python312/python.exe c:/Users/TA/Desktop/hw3/207521279-345683247.returned/reverse_mapper.py
Enter the mapped string: NSDVJRSDXV4FANW8
m[0uC1[0Hu\2$mQ[8: No mapping found]
```

And instead of 8 we will try and input: *I as mentioned above. Which didn't work. So we tried many inputs with * and we have noticed that all of them have 8 at the beginning so we tried to get into tools using the password we got and a * at the end of the string and we got in:



Which means that we need to change the mapping file to fit the exercise requirement and make a special mapping for 8 specifically.

Ascii for 8 is 56, ascii for * is 42 so we added to our table: {56,42}.

Which gives us the correct mapping:

```
/Users/TA/Desktop/hw3/207521279-345683247.returned/reverse_mapper.py
Enter the mapped string: NSDVJRSDXV4FANW8
m[0uC1[0Hu\2$mQ*
```

Part 2

Running client.exe:

```
What would you like to do?
[1] ECHO - ping the server with a custom message, receive the same.
[2] DMSG - Download message from the server.
[3] TIME - Get local time from server point of view.
[4] HMKH - Request a spinning top for Hanukkah!
your choice (4 letters command t): DMSG
4925-57467+867+86Q22-274686528-87266+96262657227-7636A74-47469766527+723-3486526-6697
323-368656K6423-36968+629-9433A28+6
496625-56667+87228-87368+765+86522-272656A7367+867+724-4776523-3736866+9756K6423-3667
2656523-374686525-56775792K
67+866+86528-864+975737428-8666965+96427-76A24-46366+9646523-36A737366+963696A7465642
3-37769746823-374686525-55248+74242455256+9434A56-6545552454425-565766565+97428+6
57686568+627-77468697328-86368+7646528-8697322-2757365642K27-77266+96K6K6967+76723-37
4686526-66469636523-3736868+7756K6425-5726573756K7429-97769746829-9637562657326-67468
6A7425-5737567+629-97467+822-27365766566+828+6

4767+8626K6966+8
```

Trying to run secure_pipe:

```
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./secure_pipe.exe "AAA"
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> ./secure_pipe.exe
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned> 
```

Opening secure_pipe on IDA.

We find some randomizing calls in sub_401410:



Which reminds us of hw1 function calls (and also function address is similar. And the usage of srand gives a hint of high likelihood of some encryption). This leads us to believe that some input might be randomized.

After asking on discord we have been told to statically analyze the file so we do.

(sub 401410:)

```
loc_4014E9:
movsx    eax, [ebp+var_16]
cmp      eax, 0Ch          ; switch 13 cases
ja       def_4014FD        ; jumtable 004014FD default case, cases 2-9
```

We have to figure out which are the 13 cases

Based off of the data preparation stage we figured out there is an upper nibble and lower nibble which's values are being compared in a 13 case switch.

We'll mark a random number generated by rand function call (which is between 2 and 9) in 'rand':

Value (decimal)	buffer
0	'rand'+ '-' + 'rand'
1	A
10	J
11	Q
12	K
2-9	'rand_1' + '+' + 'rand_2'
Else: 'x'	x+48

First lets create the function reverse the actions made in sub401410 to figure out how to revert the string. After making a small parser we got:

```
I took the robber captive. He is held in C1.If for some reason we should free the guy,one must find a code
associated with the ROBBER_CAPTURED event.When this code is used, rolling the dice should result with cub
es that sum to seven.Goblin
```

Search for the function that prints the string in client.exe:

```
mov     eax, [ebp+Buffer]
mov     [esp+24h+Str], eax ; Buffer
call    puts
mov     eax, ds:_iob
add     eax, 20h ; ' '
mov     [esp+24h+Str], eax ; Stream
call    fflush
```

We probably to replace puts function using injector.

In the injector template we will change the args (dll path to our dll, application to the client.exe to inject our new dll, and the comand line input DMSG to automatically run the command):

```

std::string lpAppName = "client.exe";
//TODO: change name dll to fit requirements
DLL_PATH = (LPSTR)"hook_scure_pipe.dll";
std::string args = std::string(lpAppName) + " ";
const char* lpCmdLine = "client.exe DMSG";

```

We will use the workshop file to mimic IAT hooking:

```

DWORD IAT_Func_Offset = 0x0000A258; // IAT hook

```

Address was chosen based on the distance to puts relatively to base(400000):

0040A250	printf
0040A254	putchar
0040A258	puts
0040A25C	scanf
0040A260	setlocale

Now after creating the dll and the injector we got our injector.exe and we get the output:

```

PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\inject> ./ClientInjector.exe
overwriting IAT address: 0x0040a258
Injected dll successfully
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\inject>
What would you like to do?
[1] ECHO - ping the server with a custom message, receive the same.
[2] DMSG - Download message from the server.
[3] TIME - Get local time from server point of view.
[4] HNKH - Request a spinning top for Hanukkah!
your choice (4 letters command t):
I took the robber captive. He is held in C1.
If for some reason we should free the guy,
one must find a code associated with the ROBBER_CAPTURED event.
When this code is used, rolling the dice should result with cubes that sum to seven.

Goblin

```

Which means we successfully injected the new dll and hooked puts properly.

Part 3

Tried downloading files that are hinted at in the goblin's message from the last part(dice.exe and code.exe):

```

PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> ./dice.exe
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> ./dice.exe 7 3
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> ./dice.exe 7
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> ./dice.exe " "
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> ./codes.exe
Too many arguments, or some of them are missing.
Usage: C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3\codes.exe <CODE_KEY> <OLD_CODE>
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3>

```

Opening codes on ida and looking for an event called "ROBBER_CAPTURED"

Found:

```

loc_4014A1:
lea     eax, [ebp+var_9B]
mov     ebx, offset aCodesCodeForCo ; "\ncodes = [code for code in Event.query"...
mov     edx, 8Fh

```

So we probably need to search around the codes.exe file.

```

lea     eax, [ebp+var_80]
mov     [esp+8Ch+Str2], eax ; char *
mov     eax, [ebp+arg_0]
mov     [esp+8Ch+Str1], eax ; void *
call    sub_4018FA
mov     [esp+8Ch+Str2], offset Str2 ; "NO SUCH CODE"
mov     eax, [ebp+arg_0]
mov     [esp+8Ch+Str1], eax ; Str1
call    strcmp
test    eax, eax
jnz     short loc_40146A

```

```

mov     [esp+8Ch+Str1], 1 ; Code
call    exit

```

```

loc_40146A:
nop
sub     esp, 0FFFFFF80h

```

When debugging with input '1 1' we pass this strcmp and don't exit. However:

```

.text:00401477 push    esi
.text:00401478 push    ebx
.text:00401479 sub     esp, 9Ch
.text:0040147F mov     eax, [ebp+arg_8]
.text:00401482 mov     [esp+0A8h+Str2], eax ; Str2
.text:00401486 mov     eax, [ebp+arg_4]
.text:00401489 mov     [esp+0A8h+Str1], eax ; Str1
.text:0040148C call    strcmp
.text:00401491 test    eax, eax
.text:00401493 jz      short loc_4014A1

```

```

.text:00401495 mov     [esp+0A8h+Str1], 1 ; Code
.text:0040149C call    exit

```

```

.text:004014A1
.text:004014A1 loc_4014A1:
.text:004014A1 lea     eax, [ebp+var_80]

```

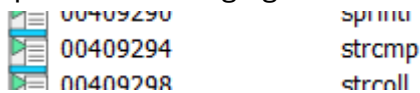
in sub401473 we fail the strcmp.

we find out that the first function uses old_code to search for an old event code and

compares it to “no such code”.

then in another function it uses strcmp which we fail. So if we bypass the strcmp check with our own implementation we could find the event required.

We will use IAT hook again with the delta address 9294, basically using the same file as in part 2 but changing minor details..:



Since we passed the first strcmp we could just check the 2nd str inputted and if the event name(str2 input) is “ROBBER CAPTURED” (as we were instructed to look for in part 2) then we could make sure the strcmp return value in EAX would be 0 (to pass the JZ condition).

After debugging with prints we found out that the first input is the event name, so we played with the order of the arguments until we got a good hook which enters the “robber captured” event, however we still got a traceback:

```
int hookFunc(const char* s1, const char* s2) {
    // printf("hookFunc called with s1: %s, s2: %s\n", s1, s2);
    if ((std::string)s2 == "ROBBER_CAPTURED") {
        printf("hookFunc called with s1: %s, s2: %s\n", s1, s2);
        return 0;
    }
    return 1;
}
```

TERMINAL

CONSOLE

gsl, LPDWORD lpThreadId);

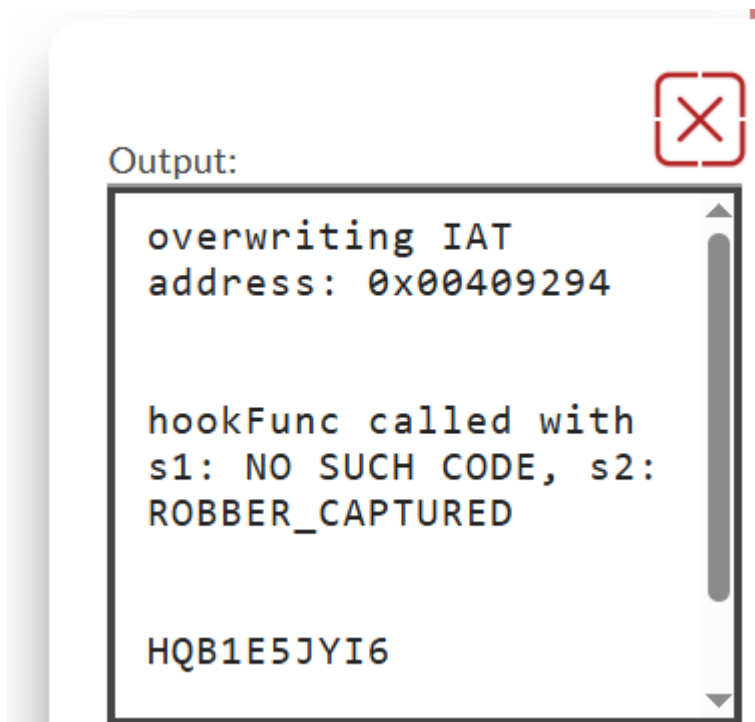
PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> ./CodesInjector.exe
overwriting IAT address: 0x00409294
Injected dll successfully

PS C:\Users\TA\Desktop\hw3\207521279-345683247.returned\part3> hookFunc called with s1: Traceback (most recent call last);, s2: ROBBER_CAPTURED
Traceback (most recent call last):

We assume we get the traceback because it needs to run on the server.



It appears that toolfix.exe is being run so we probably want to rename the inejctor to toolfix.exe.



it seems like our code is: HQB1E5JYI6.

We input the code with the 'C1' from part2 and we get halperter out of our minds:



Which is clearly not a robber...

We realize that we need to hook the function sub_401410 with new code that disables the 7 comparison for our specific code.

So we open dice.exe in CFF explorer and add a 200byte space at the end of the code.

Using the code we used in the workshop as reference we manually change the bytes in 1410 to match the jump opcode:

0	1	2	3	4	5	6	7	8
E9	EB	AB	00	00	90	90	90	90

We calculated the distance to the end of the old code:

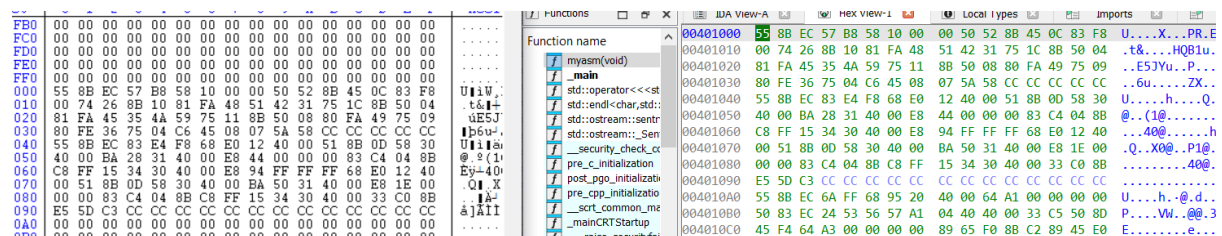
0040BFF0	?? ?? ?? ?? ?? ?? ?? ??	?? ?? ?? ?? ?? ?? ?? ??	????????????????
0040C000	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
0040C010	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
0040C020	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	

Where the section we added in CFF starts in C000 so we got:

$$C000 - 1410 - 5 = ABEB$$

Where the 5 is added for the eip after execution as we have seen in workshop.

Now all that is left is to add our function to that piece of empty space at the end of the file. We create our code in visual studio and then copy the bytes from the exe created in cff explorer:



Which gives us the new function, where we just had to adjust the return address for the jmp command(E9 XX XX XX XX):

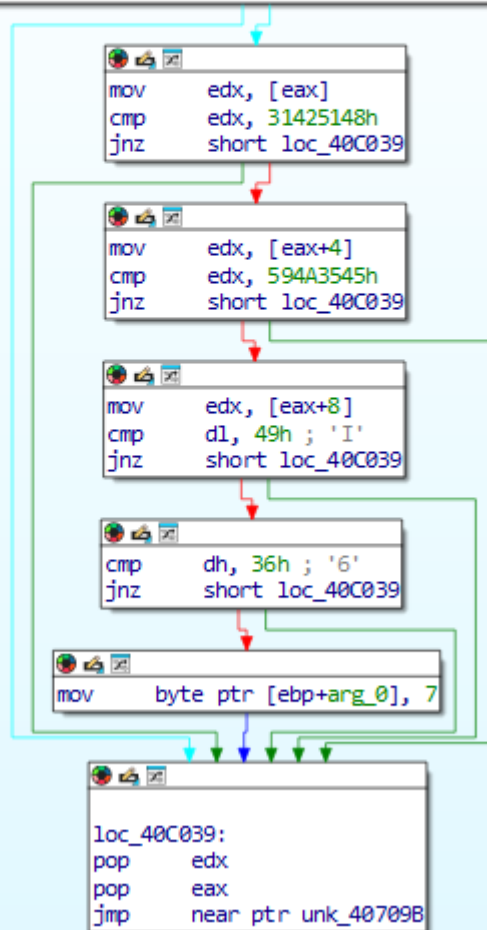
0	55	8B	EC	57	B8	58	10	00	00	50	52	8B	45	0C	83	F8
0	00	74	26	8B	10	81	FA	48	51	42	31	75	1C	8B	50	04
0	81	FA	45	35	4A	59	75	11	8B	50	08	80	FA	49	75	09
0	80	FE	36	75	04	C6	45	08	07	5A	58	E9	5B	B0	FF	FF
0	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90	90

Which copies instructions from the old code and checks for our specific dice code. Then returns to normal execution.

```

push    ebp
mov     ebp, esp
push    edi
mov     eax, 1058h
push    eax
push    edx
mov     eax, dword ptr [ebp+arg_4]
cmp     eax, 0
jz      short loc_40C039

```



And we got our robber:

